

```
TO <role>  
USING <filtering condition>  
WITH CHECK <writing condition>
```

Here, the following apply:

- `name` is the name of the policy; this is used to find it within the system.
- `table` is the table you want to apply the policy to.
- `statement` is any of `SELECT`, `UPDATE`, `DELETE`, `INSERT`, or the special keyword `ALL` to indicate all of the available statements.
- `filtering condition` is a condition used to restrict the result set of available tuples, typically, the tuples you want the role to be able to retrieve from your table.
- `writing condition` is an optional clause that provides a restriction on writing down tuples.

A policy can be removed with the `DROP POLICY` command and can be rewritten with a specific `ALTER POLICY` command.

Let's now look at a couple of examples to better understand how a policy can be built. Assume we want to allow a database user to see only the tuples in the `posts` table that belong to them. Therefore, the condition is to match the users themselves against a `SELECT` statement.

The policy could look like the following:

```
forumdb=> CREATE POLICY show_only_my_posts  
          ON posts  
          FOR SELECT  
          USING ( author = ( SELECT pk FROM users  
                           WHERE username = CURRENT_ROLE ) );  
  
CREATE POLICY
```

The policy has been named `show_only_my_posts` and acts against the `posts` table for every `SELECT` statement. A tuple will be returned in the final result set only if there is a match of the `USING` clause, which means only if the author is found in the `users` table and is the current database user.

Having created the policy does not mean that the policy is active; you need to enable it on the table it refers to with a specific `ALTER TABLE` command:

```
forumdb=> ALTER TABLE posts ENABLE ROW LEVEL SECURITY;  
  
ALTER TABLE
```

The preceding `ALTER TABLE` will enable all the policies created for such a table, in our case just one, but you have to be aware that if other policies are there, they will be activated too. There is no way to selectively enable a single policy for a table: the policies will all be enabled or disabled at once.



You must be the owner of the table in order to enable or disable RLS.

Now the role has been restricted to “see” just their own posts, but what about creating new posts? Since there is no restriction on write permissions in the policy, the user is able to create every tuple in the `posts` table. We can limit the users’ write ability, for instance, making it clear that they can only modify posts that belong to them and within a certain period of time, let’s say one day. This results in a policy such as the following:

```
forumbd=> CREATE POLICY manage_only_my_posts
           ON posts
           FOR ALL
           USING ( author = ( SELECT pk FROM users
                               WHERE username = CURRENT_ROLE ) )
           WITH CHECK ( author = ( SELECT pk FROM users
                                   WHERE username = CURRENT_ROLE )
                       AND
                       last_edited_on + '1 day'::interval >= CURRENT_
TIMESTAMP );
CREATE POLICY
```



Since RLS has already been activated for the `posts` table, the freshly created policy will be immediately active.

In this case, whatever statement the user executes against the table, they will only see their own posts (the `USING` clause) and will not be able to write (that is, `INSERT`, `UPDATE`, or `DELETE`) any tuple that does not belong to them and is not in the time range of 1 day (the `CHECK` clause).

What is happening under the hood? PostgreSQL silently applies the `USING` and `CHECK` clauses to every query you issue against the table to filter the possible tuples.

For example, if you observe the query plan of a non-filtering SELECT command, you will see that the CURRENT_ROLE filter is applied as in the USING clause:

```
forumdb=> EXPLAIN SELECT * FROM posts;

                                QUERY PLAN
-----
Seq Scan on posts (cost=8.17..76.17 rows=1000 width=74)
  Filter: (author = $0)
  InitPlan 1 (returns $0)
    -> Index Scan using users_username_key on users (cost=0.15..8.17
rows=1 width=4)
      Index Cond: (username = (CURRENT_ROLE)::text)
```

The filter has been applied by PostgreSQL even if the query does not mention it. This means that PostgreSQL is always “forced” to execute the query and filter the results for you, so you cannot expect any performance gain in using RLS. After all, the tuples must be excluded somewhere!

Now, if you try to modify the tuples in a way that violates the CHECK condition, PostgreSQL will complain and will not allow you to perform the changes:

```
forumdb=> UPDATE posts
          SET last_edited_on = last_edited_on - '2 weeks'::interval;
ERROR:  new row violates row-level security policy for table "posts"
```

You can always inspect RLS via the special \dp command in psql (the following output has been trimmed to fit the page boundaries):

```
forumdb=> \dp posts

Access privileges
|
+-----+ Policies
+-----+
| show_only_my_posts (r):
+
| (u): (author = ( SELECT users.pk
+
| FROM users
+
|
```

```

| WHERE (users.username = (CURRENT_ROLE)::text)))
+
| manage_only_my_posts:
+
| (u): (author = ( SELECT users.pk
+
| FROM users
+
| WHERE (users.username = (CURRENT_ROLE)::text)))
+
| (c): ((author = ( SELECT users.pk
+
| FROM users
+
| WHERE (users.username = (CURRENT_ROLE)::text))) AND ((last_edited_on +
'1 day'::interval) >= CURRENT_TIMESTAMP))
(1 row)

```

Lastly, you can disable or enable back policies on a table by issuing a specific `ALTER TABLE` command, such as the following:

```

forumdb=> ALTER TABLE posts DISABLE ROW LEVEL SECURITY;
ALTER TABLE

-- to enable the RLS again
forumdb=> ALTER TABLE posts ENABLE ROW LEVEL SECURITY;
ALTER TABLE

```

Policies can be combined depending on the particular statement issued by the user. By default, the policies are created as *permissive*, meaning that they are combined by means of a logical “OR.” This means that it suffices for a single policy to grant the operation in order for a statement to succeed. On the other hand, policies created as *restrictive* will be merged by means of a logical “AND,” and therefore each and every policy must be successful for the statement to be executed.

The type of permissiveness of a policy can be set only at creation time, so for example, to make the `show_only_my_posts` policy restrictive, you need to delete it and re-create it specifying its permissiveness explicitly:

```

forumdb=> DROP POLICY show_only_my_posts ON posts;
DROP POLICY

```

```

forumdb=> CREATE POLICY show_only_my_posts
           ON posts
           AS restrictive
           FOR SELECT
           USING ( author = ( SELECT pk FROM users
                               WHERE username = CURRENT_ROLE ) );

CREATE POLICY

```

Role password encryption

The login passwords associated with roles are always stored in an encrypted form, even if the role is created without the `ENCRYPTED PASSWORD` property. PostgreSQL determines the algorithm to use in order to encrypt the password via the `password_encryption` option in the `postgresql.conf` configuration file. By default, the value of the option is set to `scram-sha-256`:

```

forumdb=> show password_encryption;
password_encryption
-----
scram-sha-256
(1 row)

```

PostgreSQL introduced the SCRAM-SHA-256 encryption algorithm in version 10; before that, the encryption algorithm was set to a less robust `md5` one, which is also the only other (but now discouraged) available option.

It is important to note that you cannot change the password encryption algorithm of a live system without resetting all the passwords of the active roles. In other words, if you decide to migrate from an old `md5` to a more recent SCRAM-SHA-256 (or vice versa), you need to issue the appropriate `ALTER ROLE` statements to insert a new password for every role you have defined in the database.



Since the `pg_authid.rolepassword` field starts with the encryption algorithm, either `md5` or SCRAM-SHA-256, it is simple to inspect the system catalog and find roles that have not been updated with a new encryption algorithm.

SSL connections

The **Secure Sockets Layer (SSL)** allows PostgreSQL to accept encrypted network connections, which means every single piece of data in every packet is encrypted and therefore protected against network spoofing, as long as you handle your keys and certificates appropriately.

In order to enable the SSL extension, you first need to configure the server, then accept incoming SSL connections, and finally instrument the clients to connect in SSL mode.

Configuring the cluster for SSL

In order to let SSL do the encryption, the server must have private and public certificates. Creating and managing certificates is beyond the scope of this book and is a complex topic; you can check the PostgreSQL official documentation for the steps needed to create your own certificates. Once you or your organization have the certificates, the only thing you need to do is import the certificate and key files into your PostgreSQL server.

Assuming your certificate and key files are named `server.crt` and `server.key`, respectively, you have to configure the following parameters in the `postgresql.conf` configuration file:

```
ssl = on
ssl_key_file = '/postgres/16/data/ssl/server.key'
ssl_cert_file = '/postgres/16/data/ssl/server.crt'
```

This is done, of course, with the absolute path to your files. The first line tells PostgreSQL to enable SSL, while the other two lines tell the server where to find the files required to establish an encrypted connection. Of course, those files must be readable by the user who runs the PostgreSQL cluster (usually the `postgres` operating system user).

Once you have enabled SSL, you need to adjust the `pg_hba.conf` file to allow the host-based access machinery to handle SSL-based connections. In particular, if you don't want to accept plain connections, you need to substitute every `host` entry with `hostssl`, for instance:

```
hostssl    all    luca      venkman      scram-sha-256
hostssl    all    forum    192.168.222.1/32  scram-sha-256
```

If you want to accept both plain and encrypted connections, you can leave `host` as the connection method.

Connecting to the cluster via SSL

When connecting to PostgreSQL, the client will switch automatically to an SSL connection if the host-based access has a `hostssl` entry; otherwise, it will default to a standard plain connection.

If `pg_hba.conf` has a host line, this means that it can accept both SSL and plain connections. Therefore, you need to force the connection to be SSL when you are initiating it. In `psql`, this can only be achieved by using a connection string and specifying the `sslmode=require` parameter to enable it. The server, if accepting the connection, will report the SSL protocol in use:

```
$ psql "postgresql://forum@localhost:5432/forumdb?sslmode=require"
psql (16.0)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, bits:
256, compression: off)
Type "help" for help.

forumdb=>
```

If you omit the `sslmode` parameter or use the standard `psql` connection parameters, the connection will be turned into SSL if the `pg_hba.conf` file has a `hostssl` line that matches. For instance, the following three connections produce the same result (an encrypted connection):

```
$ psql -h localhost -U forum forumdb
psql (16.0)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, bits:
256, compression: off)
Type "help" for help.

forumdb=> \q

$ psql "postgresql://forum@localhost:5432/forumdb"
psql (16.0)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, bits:
256, compression: off)
Type "help" for help.

forumdb=> \q

$ psql "postgresql://forum@localhost:5432/forumdb?sslmode=require"
psql (16.0)
```

```
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, bits:
256, compression: off)
Type "help" for help.

forumdb=>
```

Similarly, you can specify that you don't want an SSL connection at all by setting `sslmode=disable`. This time, if `pg_hba.conf` has a `hostssl` mode, the connection will be rejected, while it will be served as a non-encrypted one if the `pg_hba.conf` file has a `host` line:

```
$ psql "postgresql://forum@localhost:5432/forumdb?sslmode=require"
psql: error: could not connect to server: FATAL: no pg_hba.conf entry for
host "127.0.0.1", user "forum", database "forumdb", SSL off
```

From the error, you can clearly see that there is no line that accepts a plain (host mode) connection in the `pg_hba.conf` file, or, on the other hand, that there are only `hostssl` lines.

Lastly, if you require the connection to use SSL but the PostgreSQL server is not configured to use SSL, an error message about the mismatch will be reported:

```
$ psql "postgresql://forum@localhost:5432/forumdb?sslmode=disable"
psql: error: could not connect to server: server does not support SSL, but
SSL was required
```

Summary

In this chapter, we learned that PostgreSQL provides a very rich infrastructure for managing permissions associated with roles. Internally, PostgreSQL handles permissions for different database objects by means of ACLs, and every ACL contains information about the set of permissions, the users to whom permissions are granted, and the user who granted such permissions. In terms of tabular data, it is even possible to define column-based permissions and row-level permissions to exclude users from having access to particular subsets of data.

Permissions are granted by nested roles in a dynamically inherited way or on demand, leaving you the option to fine-tune how a role should exploit privileges.

Lastly, when opportunely configured, a server can handle network connections via SSL, thereby encrypting all network traffic and data.

In the next chapter, you will learn all about transactions and how PostgreSQL manages them in a concurrent scenario, providing rock-solid stability to your data.

Verify your knowledge

- What is a role?

A role can be a single user or a group of users that have access to the cluster and its databases. A role is the basic unit to grant access and define permissions. See the *Understanding roles* section for more details.

- What does the INHERITS clause do?

The INHERITS clause makes a role inherit, that is get, instantly and dynamically all the permissions granted to the role from which it inherits. Without the INHERITS clause, the role still has the permissions of the role it belongs to, but an explicit SET ROLE is required in order to use such permissions. See the *Roles that inherit from other roles* section for more details.

- What is an **Access Control List (ACL)**?

An ACL is the specification of a set of permissions attached to a database object, and is the way PostgreSQL implements and store the permissions. See the *ACLs* section for more details.

- What are the statements to add a permission to a role or remove a permission from a role?

The GRANT statement adds a permission to a role, while the REVOKE statement removes a permission from a role. The special keyword ALL can be used to grant or revoke all the available permissions for the object, while multiple permissions can be specified at once in both the commands. See the *Granting and revoking permissions* section for more details.

- What is **Row-Level Security (RLS)**?

RLS is a way to restrict the result set of a query depending on the role that is executing it. RLS can be applied to both read queries and write queries. See the *RLS* section for more details.

References

- CREATE ROLE statement official documentation: <https://www.postgresql.org/docs/current/sql-createrole.html>
- ALTER ROLE statement official documentation: <https://www.postgresql.org/docs/current/sql-alterrole.html>

- DROP ROLE statement official documentation: <https://www.postgresql.org/docs/current/sql-droprole.html>
- GRANT statement official documentation: <https://www.postgresql.org/docs/current/sql-grant.html>
- REVOKE statement official documentation: <https://www.postgresql.org/docs/current/sql-revoke.html>
- PostgreSQL pg_roles catalog details: <https://www.postgresql.org/docs/current/view-pg-roles.html>
- PostgreSQL pg_authid catalog details: <https://www.postgresql.org/docs/current/catalog-pg-authid.html>
- PostgreSQL ACL documentation: <https://www.postgresql.org/docs/current/ddl-priv.html>
- PostgreSQL host-based access rule details: <https://www.postgresql.org/docs/current/auth-pg-hba-conf.html>
- PostgreSQL ACL utility functions: <https://www.postgresql.org/docs/current/functions-info.html>

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow the QR code below:

<https://discord.gg/jYWCjF6Tku>

